



UNIVERSIDAD
TORCUATO DI TELLA

Licenciatura en Tecnología Digital

Tecnología Digital VI: Inteligencia Artificial

Procesamiento de texto

Clase práctica 9

Motivación

- ¿Qué es el **NLP**?
- ¿Cómo podemos **incorporar texto** a modelos como los que vimos hasta ahora?
- Al hacerlo, ¿con qué **problemas** nos podemos enfrentar? ¿Cómo los podemos **solucionar**?

Procesamiento del lenguaje natural

- En inglés, se lo conoce como *natural language processing* (**NLP**).
- Enfocada en examinar cómo **estudiar con técnicas computacionales el lenguaje humano**.
- A modo de ejemplo, algunas aplicaciones son:
 - Realizar un análisis automático de **entrevistas** para predecir en jóvenes la posterior aparición de **psicosis**
 - Analizar el humor del contenido textual en **Twitter** para predecir el valor de cierre de un **índice** **bursátil**

Un primer panorama

Terminología:

- Colección de textos: corpus.
- Un elemento del corpus: documento.
- Documentos pueden tener asociados ciertos **metadatos** (i.e., datos sobre los datos, como fecha de creación o autor).

¿Cómo podemos representar un texto para poder introducirlo en algoritmos de aprendizaje automático?

- Usar algoritmos que puedan tomar secuencias como *input*.
- Transformar el texto en una matriz. Hoy, nosotros vamos a ver esta opción.

Entonces, el **objetivo** es representar al corpus (i.e., la colección de textos) como una **matriz de números** (para así poder aplicar las técnicas vistas en la materia).

Para ello, vamos a usar lo que se conoce como el **bag-of-words model**. Esta es una de las múltiples formas de abordar el problema.

Pero, antes de ver dicho modelo, necesitamos entender la tokenización.

Tokenización

- Los seres humanos entendemos a un texto como una **secuencia de palabras**.
- Pero, un texto no es más que una **secuencia de caracteres** y a la computadora le da lo mismo el carácter espacio que el carácter a.
- **Tokenizar** es la acción de **dividir un conjunto de caracteres en una secuencia de palabras o tokens**.

=)Feeeliiciidaadeeeess !! (: k t lo pases muy bien!!

Tokenización (continuación)

En la práctica lo más común es usar librerías que, entre otras cosas, tienen implementados, para distintos idiomas, lo que se conoce como **tokenizadores**.

Algunas de estas librerías son [NLTK](#), [TextBlob](#) y [spaCy](#).

Cabe aclarar que es común que **uno deba adaptar** estos tokenizadores al **dominio en que se está trabajando**.

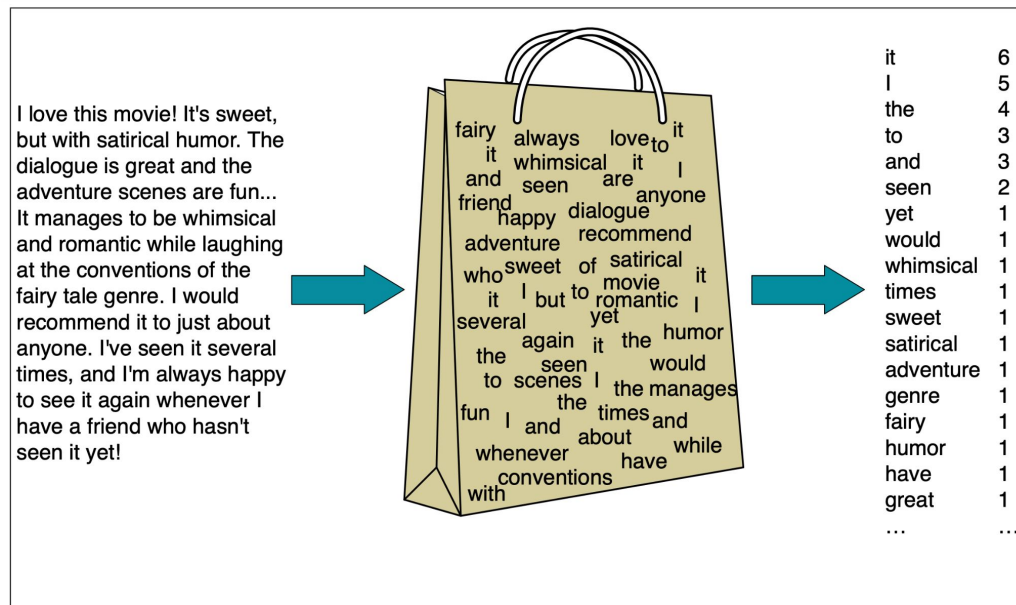
Procesamiento previo

Al tokenizar y usar el bag-of-words model, se suele hacer un **preprocesamiento de los textos**.

Concretamente, algunas de las **operaciones** que se suelen hacer son:

- Pasar todo a **minúscula**.
- Quitar **stopwords** / palabras vacías (usadas individualmente, sin acompañar a las palabras claves del texto, carecen de sentido; e.g., **e1**, **1a**).
- Ignorar palabras (muy) **poco frecuentes**.
- Asignar etiquetas **part-of-speech** a las palabras. E.g., **banco** como sustantivo vs. **banco** como verbo (sinónimo de apoyar, hacer el aguante).
- Reducir las palabras a su **forma común**.
 - **Stemming**: lo hace aplicando heurísticas que quitan el final de la palabra. De **having** a **hav**.
 - **Lematización**: lo hace de manera más apropiada usando vocabulario y análisis morfológico (vinculado a la estructura de las palabras). De **having** a **have**.
 - En Python, existen múltiples **librerías** para hacer esta reducción. Por ejemplo, [Stanza](#).

Bag-of-words model | Ilustración



Contar la frecuencia de cada palabra en el documento

Figure 4.1 Intuition of the multinomial naive Bayes classifier applied to a movie review. The position of the words is ignored (the *bag-of-words* assumption) and we make use of the frequency of each word.

Bag-of-words model | Matriz documento-término

La matriz documento-término (**DTM**, por las siglas en inglés: *document-term matrix*) tiene:

- Tantas **filas** como **documentos** distintos tenga el corpus.
- Tantas **columnas** como **tokens** distintos tenga el corpus.

	battle	good	fool	wit
As You Like It	1	114	36	20
Twelfth Night	0	80	58	15
Julius Caesar	7	62	1	2
Henry V	13	89	4	3

Obras de Shakespeare
(documentos)

A la DTM se la puede usar para entrenar modelos.

Bag-of-words model | Algunos de sus problemas

- Si el vocabulario es grande, hay **muchas columnas**.
- **Perdemos toda la información referida al orden de las palabras**. ¿toy dog (perro de juguete) = dog toy (juguete para perros)?
- **Ignora el contexto de las palabras, como el que estén negadas**. ¿La comida era rica = La comida no era rica?
- **Ignora la similitud semántica entre palabras**. En consecuencia, **palabras que significan lo mismo pueden terminar en columnas distintas**. ¿Auto != Automóvil? ¿Encendedor != Mechero?
- No contempla la **polisemia** (i.e., pluralidad de significados de una expresión lingüística; [fuente](#)).

Bag-of-words model | Vectores

Los documentos pueden ser pensados como vectores. De ahí, el nombre *vector space models*.

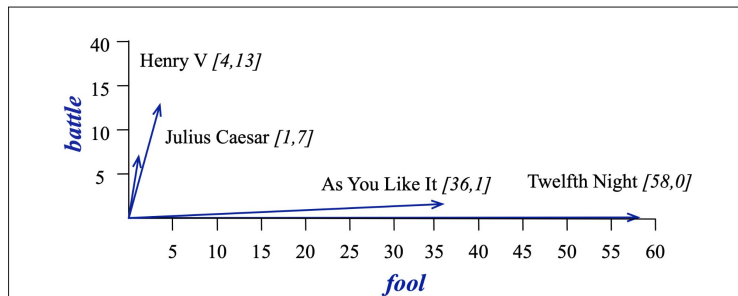


Figure 6.4 A spatial visualization of the document vectors for the four Shakespeare play documents, showing just two of the dimensions, corresponding to the words *battle* and *fool*. The comedies have high values for the *fool* dimension and low values for the *battle* dimension.

Mirando este gráfico, intuitivamente, ¿cuándo podríamos decir que dos documentos son **similares**?

Cuando **apuntan en la misma dirección**.

Una forma de medir qué tanto dos documentos apuntan en la misma dirección es mediante la **similitud coseno**.

Similitud coseno | Concepto

Es el **coseno del ángulo entre dos vectores**.

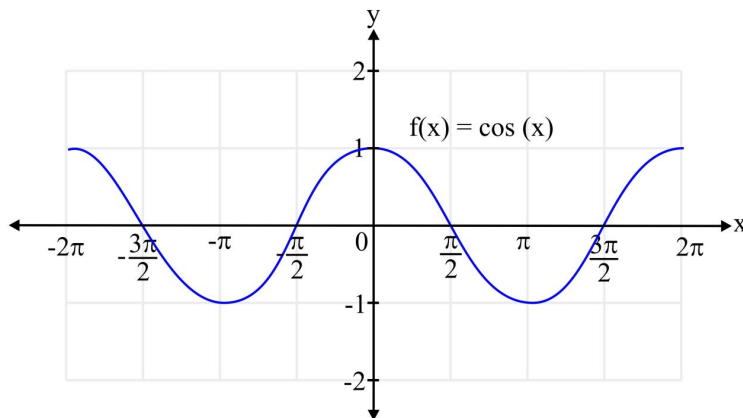
Se basa en el **producto punto**, también llamado producto interno. Este producto funciona como una **métrica de similitud** porque tiende a ser alto cuando los dos vectores tienen valores grandes en las mismas dimensiones. En cambio, vectores que tienen ceros en diferentes dimensiones (vectores ortogonales) tienen un producto punto igual a 0, representando una fuerte disimilitud.

$$\text{dot product}(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i = v_1 w_1 + v_2 w_2 + \dots + v_N w_N$$

Por otra parte, este producto es mayor si un vector es más largo, con altos valores en cada dimensión. Este es el caso de **palabras frecuentes**, cuyo producto punto es mayor. Esto es un **problema**, porque queremos saber cuán similares son dos palabras indistintamente de su frecuencia. En consecuencia, al producto punto se lo divide por la longitud de cada uno de los dos vectores. Este producto punto **normalizado** es igual al coseno del ángulo entre los dos vectores.

$$\frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| |\mathbf{b}|} = \cos \theta$$

Similitud coseno | Concepto (continuación)



En este contexto, ¿puede la similitud coseno ser negativa?

No, porque los números de la matriz son siempre 0 o positivos. El coseno vale **1** para vectores apuntando en la misma dirección, **0** para vectores ortogonales y **-1** para vectores apuntando en direcciones opuestas. **Pero**, como las **frecuencias son valores no-negativos**, el coseno para los vectores en cuestión oscila entre 0 y 1 (ambos extremos incluidos).

Similitud coseno | Efecto de las stopwords

Stopwords: palabras que ocurren muy frecuentemente y que, dependiendo del contexto, aportan poca información.

¿**Cómo afectan** estas palabras a la **similitud coseno** entre los documentos?

Mueven mucho los vectores, haciendo que tiendan a apuntar en direcciones similares cuando, en realidad, las palabras claves de cada uno no necesariamente son tan similares.

Una posible **solución** es **quitarlas**, como vimos antes con el preprocesamiento.

Otra opción es usar la **normalización TF-IDF**.

Normalización TF-IDF

Es el **producto de 2 términos**:

- **Frecuencia de la palabra (TF)**. La frecuencia de la palabra t en el documento d .
 - **Le quita peso a una palabra excesivamente frecuente en un documento.**
 - El que una palabra aparezca 100 veces en un documento no hace que esa palabra sea 100 veces más relevante para el significado del documento.

$$\text{tf}_{t,d} = \log_{10}(\text{count}(t,d) + 1)$$

Para sacarle peso a las palabras que aparecen muchas veces en el documento

Normalización TF-IDF (continuación)

- Frecuencia inversa del documento o peso idf de la palabra (**IDF**). N es el número total de documentos en el corpus. df_t es el número de documentos en que ocurre la palabra t .
 - Le quita peso a una palabra que aparece en todos los documentos.
 - En otras palabras, se incorpora para dar mayor peso a palabras que ocurren sólo en unos pocos documentos, ya que son más útiles para discriminar esos documentos del resto, relativo a términos que ocurren frecuentemente a lo largo de todo el corpus.
 - A menor cantidad de documentos en que esta palabra ocurre, mayor es este peso.

$$idf_t = \log_{10} \left(\frac{N}{df_t} \right)$$

Normalización TF-IDF (continuación)

Entonces, el producto de estos 2 términos le da **mayor importancia** a las palabras que **aparecen muchas veces y en pocos documentos**.

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

6.9 normaliza las apariciones

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.2 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.074	0	0.22	0.28
good	0	0	0	0
fool	0.019	0.021	0.0036	0.0083
wit	0.049	0.044	0.018	0.022

Figure 6.9 A tf-idf weighted term-document matrix for four words in four Shakespeare plays, using the counts in Fig. 6.2. For example the 0.049 value for *wit* in *As You Like It* is the product of $\text{tf} = \log_{10}(20 + 1) = 1.322$ and $\text{idf} = .037$. Note that the idf weighting has eliminated the importance of the ubiquitous word *good* and vastly reduced the impact of the almost-ubiquitous word *fool*.

Reseñas de Películas

Supongamos que tenemos un **dataset de reseñas de películas**. Para cada reseña tenemos el **texto y su ranking** (del 1 al 10). Si quisieramos solamente con el texto **predecir que ranking** le otorgo un usuario que podríamos hacer aplicando lo que vimos hoy de procesamiento de texto?

O bien generamos features manualmente. Es decir, creo variables como por ejemplo: variable binaria si la palabra “like” esta en el texto o no, longitud del texto, emoji positivo o negativo, etc.

O bien procesamos con bag of words por ejemplo!

Y que obtenemos?

Reseñas de Películas

Supongamos que tenemos un **dataset de reseñas de películas**. Para cada reseña tenemos el **texto y su ranking** (del 1 al 10). Si quisieramos solamente con el texto **predecir que ranking** le otorgo un usuario que podríamos hacer aplicando lo que vimos hoy de procesamiento de texto?

O bien generamos **features manualmente**. Es decir, creo variables como por ejemplo: **variable binaria si la palabra “like” esta en el texto o no, longitud del texto, emoji positivo o negativo, etc.**

O bien procesamos con **bag of words** por ejemplo!

Y que obtenemos? Una matriz de **reseñas (docs) vs tokens (términos)**

Posibles problemas

- ¿Cual es la forma y tamaño resultante de la matriz?

Matriz muy ancha ($p > n$) y esparsa (porque puede haber palabras poco comunes)

- ¿Qué problemas vienen de la mano de la pregunta anterior?

El costo

- ¿Y si quisiera visualizar la relación entre documentos y/o rankings?

Muy engorroso

- ¿Conocemos alguna técnica que nos ayude a reducir el tamaño “sin perder demasiada información”?

PCA

Posibles soluciones

Podemos probar PCA y/o t-SNE entre otros, aunque **al ser matrices esparsas y de muchas dimensiones** es probable que necesitemos **reducir manualmente antes un poco estas “columnas candidatas”**.

T-distributed Stochastic Neighbor Embedding.

t-SNE [1] is a tool to visualize high-dimensional data. It converts similarities between data points to joint probabilities and tries to minimize the Kullback-Leibler divergence between the joint probabilities of the low-dimensional embedding and the high-dimensional data. t-SNE has a cost function that is not convex, i.e. with different initializations we can get different results.

It is highly recommended to use another dimensionality reduction method (e.g. PCA for dense data or TruncatedSVD for sparse data) to reduce the number of dimensions to a reasonable amount (e.g. 50) if the number of features is very high. This will suppress some noise and speed up the computation of pairwise distances between samples. For more tips see Laurens van der Maaten's FAQ [2].

Es probable que TruncatedSVD se porte mejor que PCA. PCA densifica la matriz y no explota en memoria y costo computacional rápidamente. Si les interesa, hay todo un campo sobre Latent Semantic Analysis (LSA)

Reseñas de Películas - PCA y t-SNE

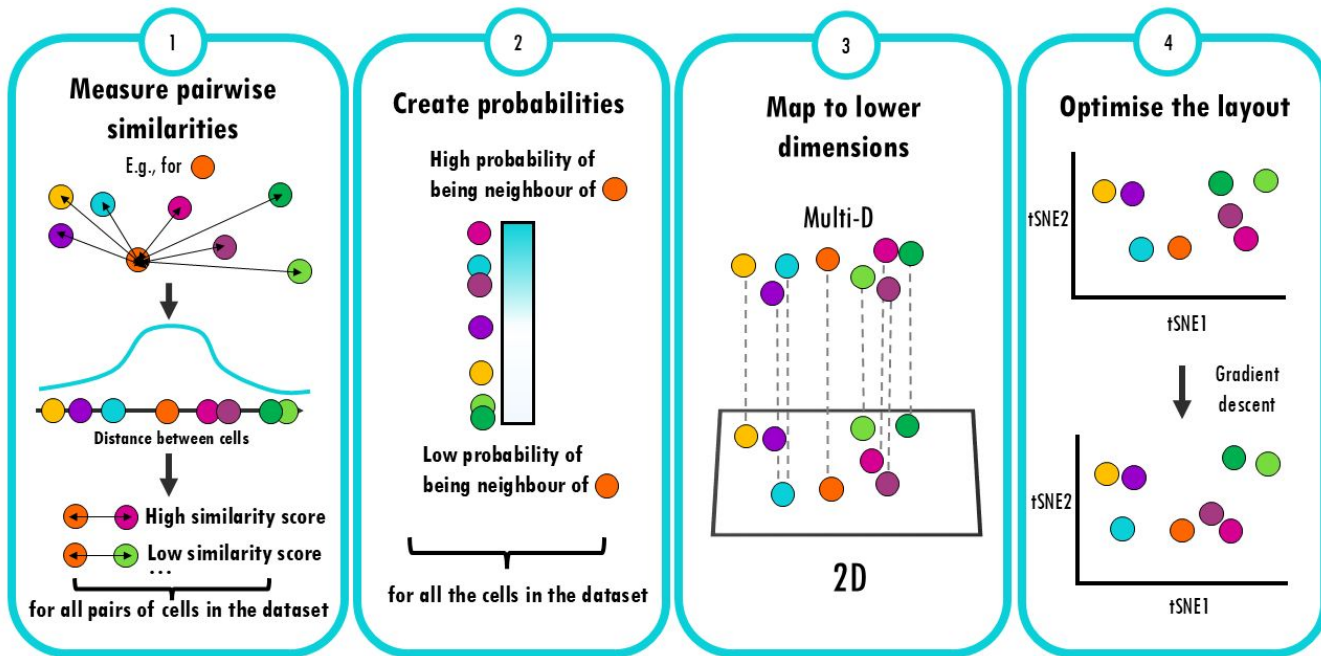
Que obtenemos para PCA?

Componentes ortogonales entre sí que serán básicamente una combinación lineal de los pesos de distintos tokens del bag of words para explicar una reseña.

Y para t-SNE?

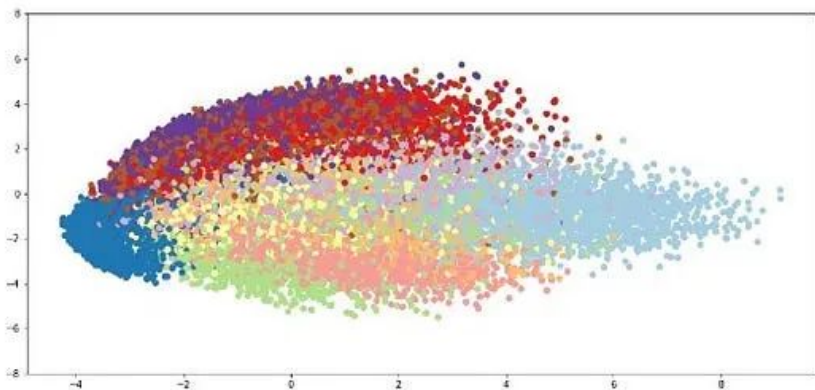
Breve concepto de t-SNE

How does t-SNE summarise many dimensions into 2?

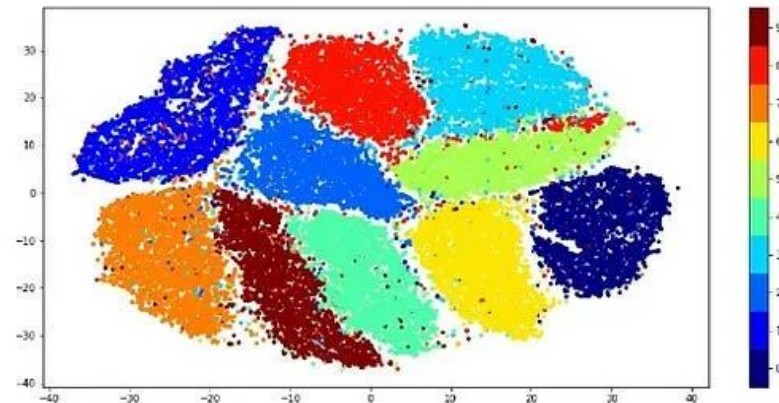


Breve concepto de t-SNE

MNIST - PCA



MNIST - TSNE



No es una fórmula cerrada ni es determinístico, pero funciona mejor para este caso.

Reseñas de Películas - PCA y t-SNE

Que obtenemos para PCA?

Componentes ortogonales entre sí que serán básicamente una combinación lineal de los pesos de distintos tokens del bag of words para explicar una reseña.

Y para t-SNE?

Menor dimensionalidad **fácil de visualizar vecindarios locales**. Sin garantías ni interpretabilidad en los ejes que obtengamos.

Reseñas de Películas

Si quisieras predecir el ranking cual usarías? **Depende**

- **PCA engloba mayor varianza** en primeros componentes y nos da interpretabilidad de cada uno, pero no nos garantiza que sean “buenos predictores” del ranking, estamos perdiendo información.
- **t-SNE nos da lindas visualizaciones**, pero nos hace perder información respecto a las distancias originales entre “clusters”. Rompe estructura global.
- no aplicar ninguna no nos hace perder información, pero habría que **manejar un dataframe esperso**, es decir, lleno de ceros y muy ancho en este sentido.

Reseñas de Películas

La mejor respuesta al problema de predicción está en **experimentar conscientemente** de lo que hacemos, **evaluar** que funciona mejor y tratar de **entender el porqué** puede estar pasando.

